THE PRAGMATIC DBMS IMPLEMENTATION ROADMAP

Maximizing Deep Architectural Mastery While Eliminating Low-Level Toil

Focus: High-Leverage Systems Concepts & Real Architectural Intuition
Courses: CMU 15-445 (Core Relational) & CMU 15-721 (Modern OLAP / DuckDB)
Workspace: /home/omar/Learning Projects/db-internals

The 80/20 Rule: Focus 100% on core algorithms, memory hierarchy, and concurrency.
Toil Elimination: Strip raw SQL lexers, variable-length defragmentation, and LLVM bytecode.
Baseline Status: Disk Manager (Done) • Buffer Pool (Done) • B+ Tree (Done)

1. THE 80/20 LEARNING PHILOSOPHY: TOIL VS. DEEP INSIGHT

The Systems Engineering Paradox in Database Learning

Building a database from scratch is universally considered the best way to understand systems programming. However, many engineers fall into the **'Low-Level Toil Trap'**: spending weeks debugging byte alignment for variable-length strings or writing recursive-descent SQL parsers. These exercises teach compiler mechanics and pointer math, **not database architecture**. By skipping or simplifying low-level toil, you reach the high-leverage concepts—Volcano execution, Hash Joins, Cost-Based Access Path Selection, MVCC, and Vectorized SIMD processing—in a fraction of the time.

Component / Topic	What Takes Weeks (Low-ROI Toil)	What Gives 90% Conceptual Mastery (High-ROI Core)	Pragmatic Decision
Record Storage & Tuples	Writing custom byte serializers for 10 SQL data types, null bitmaps, endianness, and in-page slot compaction.	Fixed-size records (e.g. struct { int64_t id; char name[32]; }) + (Key -> RID) index hookup.	**SIMPLIFY** **Fixed Tuples**
SQL Parsing & Binding	Writing a Lexer, recursive-descent Parser, and AST visitor from scratch (Compilers 101).	Constructing query plans directly in C++ code: HashJoin(SeqScan(), IndexScan()).	**SKIP PARSER** **Programmatic Plan**
Volcano Execution & Joins	Over-engineering abstract type polymorphism and 20 SQL scalar functions.	The Init()/Next() pull pipeline, **Hash Join** (Build vs Probe), and sequential/index scans.	**MUST BUILD** **Pure Gold**
Optimizer & Access Path	Full dynamic programming join enumeration for 10+ table joins.	**Access Path Selection**: understanding why and when the DB chooses an Index Scan vs a Sequential Scan based on selectivity.	**MUST BUILD** **High Insight**
Transactions & MVCC	Writing a complex multi-granularity lock escalation table with strict deadlock graph traversal.	**MVCC Snapshot Isolation** (xmin, xmax): understanding why readers never block writers and writers never block readers.	**MUST BUILD** **Modern Core**
Crash Recovery & ARIES	Implementing multi-pass ARIES Undo with Compensation Log Records (CLRs) across non-quiescent checkpoints.	The **WAL Flush Invariant** (page_lsn <= flushed_lsn) and Redo-only crash recovery replay.	**SIMPLIFY** **Redo-Only WAL**
Modern OLAP (15-721)	Emitting raw LLVM IR machine code (compilation latency often exceeds execution time).	**Vectorized Execution (DuckDB Model)**: processing batches of 1024 integers with SIMD filter pushdown.	**MUST BUILD** **50x Speedup**

Strategic Takeaway

The Verdict on Your Progress: You have already conquered the hardest, lowest-level foundation of the entire system: the **Buffer Pool Manager with LRU-K** and the **B+ Tree with Latch Crabbing**. Do not lose momentum in tedious bit-packing. By adopting simplified fixed-size records and programmatic query plans, you can build a functioning Volcano execution engine with Hash Joins and MVCC in days rather than months!

2. PRAGMATIC SYSTEM ARCHITECTURE BLUEPRINT

The Lean Database Anatomy

Instead of 10 heavily coupled layers with custom compilers and complex recovery managers, the pragmatic architecture isolates the database engine into **6 high-ROI modules**. Notice how the B+ Tree you already built connects directly to the Table Heap and the Execution Engine:

Layer 6: Modern OLAP Engine (CMU 15-721)	**Vectorized Batch Processing (DuckDB Model)**: Operators process VectorBatch[1024] with SIMD filter pushdown (AVX2/AVX-512) and morsel-driven work-stealing parallelism.
Layer 5: Concurrency & Transactions	**MVCC Snapshot Isolation**: Tuples carry (xmin, xmax). Transactions read consistent snapshots without acquiring read locks. Write-Ahead Logging (WAL) ensures durability.
Layer 4: Execution Engine (Volcano)	**Iterator Pipeline (Init/Next)**: SeqScan, IndexScan, HashJoin (Build & Probe), and Group-By Aggregations. Direct programmatic C++ query plans.
Layer 3: Table Storage & Index Hookup	**Fixed-Size TableHeap**: 4KB pages storing fixed records. Record IDs (RID = page_id : slot) stored as values in your **B+ Tree**: Index.Insert(Key, RID).
Layer 2: Memory & Buffer Pool [DONE]	**4KB Page Frames: LRU-K (K=2) eviction, zero-copy pointer casting over page->GetData(), pin/unpin tracking.**
Layer 1: Storage & OS Interface [DONE]	**POSIX Block I/O + io_uring: Synchronous pread/pwrite and Linux io_uring asynchronous batched queues.**

The Golden Invariant: How the B+ Tree Connects to Table Records

A common point of confusion when learning databases is: *'What does the B+ Tree actually store?'*
In your single-page and multi-level tests, your B+ Tree stored int64_t Key -> int64_t Value. In a relational engine, the Value is simply a **Record ID (RID)**: struct RID { page_id_t page_id; uint32_t slot; }.
1. When inserting a row: TableHeap::Insert(row, &rid;) writes the row to a table page and returns its RID.
2. Then: Index::Insert(row.id, rid) inserts the key and its physical record pointer into the B+ Tree.
3. When querying WHERE id = 42: the execution engine searches the B+ Tree to find RID, then calls TableHeap::GetTuple(rid). That is the entire secret of secondary database indexing!

Engineering Shortcut

The 'Zero-Toil' Table Layout: Avoid writing variable-length slotted page defragmenters. Define a fixed row struct: struct Record { int64_t id; char name[32]; double balance; int64_t xmin; int64_t xmax; };. A 4KB page holds approximately (4096 - header) / sizeof(Record) records. This unlocks 100% of relational query execution, joins, and MVCC with zero pointer math bugs!

3. PHASE 1: PRAGMATIC CORE RELATIONAL ENGINE (CMU 15-445)

Sprints 1.1 through 1.4: Storage Hookup, Volcano Execution & Joins

SPRINT 1.1: STORAGE CONSOLIDATION & RAII PAGE GUARDS

COMPLETED / HARDENING

Prerequisite Lectures: CMU 15-445 Lecture #03 & #04
Primary Learning Goal: Harden your BufferPoolManager and B+ Tree with clean RAII memory safety before adding higher layers.

[BUILD - High ROI Core]:
• Implement PageGuard, ReadPageGuard, and WritePageGuard RAII wrappers around Page*.
• Automatically invoke bpm->UnpinPage(page_id, is_dirty) and page->RUnlatch() / WUnlatch() in the destructor.
• Eliminates 100% of buffer pool frame leaks and dangling latches during exception or early return paths.

[SKIP / SIMPLIFY - Low ROI Toil]:
• Do not write custom memory allocators or specialized disk defragmenters. Rely on your existing POSIX/io_uring disk manager.

Conceptual Takeaway: *Manual unpinning is the #1 source of database deadlocks and out-of-memory crashes. RAII guarantees leak-free execution.*

Validation Test: Write PageGuardTest: scope-exit assertions verifying pin count drops back to zero.
Deliverable Checkpoint: Bulletproof RAII memory guard wrapping BufferPoolManager frames.

SPRINT 1.2: FIXED-SIZE TABLEHEAP & B+ TREE TUPLE HOOKUP

NEXT UP (CORE PRIORITY)

Prerequisite Lectures: CMU 15-445 Lecture #03 (Database Storage: Files, Pages, Tuples)
Primary Learning Goal: Connect your existing B+ Tree to a real persistent table storage heap without variable-length byte-packing toil.

[BUILD - High ROI Core]:
• Define fixed-size record schema: struct UserRow { int64_t id; char name[32]; double score; int64_t xmin; int64_t xmax; };.
• TablePage: 4KB page frame managing an array of fixed records + a 64-bit occupancy bitmap (1 = allocated, 0 = free slot).
• RID: uint64_t bit-packed into (page_id: 32 bits, slot_num: 32 bits).
• TableHeap: Doubly linked list of TablePages (first_page_id, next_page_id). Implements Insert(row, &rid;), Get(rid, &row;), Update(rid, row), and MarkDelete(rid).
• **Index Hookup**: In your test harness, instantiate BPlusTree. When inserting a row, insert (row.id, rid) into the tree!

[SKIP / SIMPLIFY - Low ROI Toil]:
• Skip polymorphic Type and Value classes (INTEGER, VARCHAR, etc.). Use native C++ types (`int64_t`, `double`, `char[]`).
• Skip slotted-page compaction and defragmentation (fixed slots eliminate internal fragmentation entirely).

Conceptual Takeaway: *This turns your standalone key-value B+ Tree into a true database index that routes queries to physical disk records.*

Validation Test: TableHeapTest: Insert 10,000 rows into TableHeap, index on id with B+ Tree, query random IDs via B+ Tree, fetch row from TableHeap, and assert 100% match.
Deliverable Checkpoint: Fully integrated Table Storage Heap connected to concurrent B+ Tree index.

SPRINT 1.3: THE VOLCANO EXECUTION ENGINE & RELATIONAL JOINS

CORE PRIORITY

Prerequisite Lectures: CMU 15-445 Lecture #13 (Query Execution) & Lecture #12 (Join Algorithms)
Primary Learning Goal: Master the iterator execution model and build the most important analytical algorithm in history: the Hash Join.

[BUILD - High ROI Core]:
• AbstractExecutor: Pure virtual interface defining Init(), Next(UserRow* out_row, RID* out_rid) -> bool.
• SeqScanExecutor: Sequential scan traversing TableHeap pages and yielding live (non-deleted) rows.
• IndexScanExecutor: Traverses B+ Tree index using point search or range iterator, fetches matching RIDs, and loads tuples from TableHeap in O(log N) time.
• FilterExecutor: Evaluates a predicate lambda (e.g. [](const UserRow& r) { return r.score > 80.0; }) over child executor.
• **HashJoinExecutor (Equi-Join)**:
• *Build Phase*: In Init(), drains the inner child executor into an in-memory hash table (std::unordered_multimap).
• *Probe Phase*: In Next(), pulls rows from the outer child, probes the hash table, and yields matched combinations.

[SKIP / SIMPLIFY - Low ROI Toil]:
• Skip writing a dynamic expression evaluator or AST parser. Use C++ lambdas for filter conditions and join keys.
• Skip 3-way external merge sort and disk-spilling joins for now (focus on in-memory hash tables).

Conceptual Takeaway: *Volcano execution powers SQLite, Postgres, and MySQL. The Hash Join eliminates O(N × M) quadratic explosions and is the workhorse of relational databases.*

Validation Test: HashJoinTest: Join 5,000 customers with 20,000 transactions; verify Hash Join produces exact matching rows in milliseconds compared to seconds for Nested Loop Join.
Deliverable Checkpoint: Working Volcano iterator engine executing scans, filters, and high-speed hash joins.

SPRINT 1.4: COST-BASED OPTIMIZER & DYNAMIC PROGRAMMING (DP) JOIN ORDERING

CORE PRIORITY

Prerequisite Lectures: CMU 15-445 Lecture #15 & #16 (Query Optimization Part 1 & 2)
Primary Learning Goal: Master cost modeling, single-table access path selection, and multi-table dynamic programming join ordering (Selinger algorithm).

[BUILD - High ROI Core]:
• Table Statistics Tracker: Maintain row count N, page count P, and distinct key count V(A) for indexed columns.
• Cost Modeling Formulas:
• **SeqScan Cost**: C_seq = P × Cost_page_seq + N × Cost_cpu_tuple.
• **IndexScan Cost**: C_idx = (h + ceil(sel × P_leaf)) × Cost_page_seq + (sel × N) × Cost_page_random + (sel × N) × Cost_cpu_tuple.
• **Join Cost Models**: NestedLoopJoin (O(|Outer| × |Inner|)) vs HashJoin (O(|Outer| + |Inner|)).
• Access Path Selector: Compare C_seq vs C_idx based on selectivity sel (crossover at ~25-30%).
• Rule-Based Filter Pushdown: Push filter predicates below joins directly into base scan operators.
• **Selinger Dynamic Programming (DP) Join Ordering**: Enumerate all (2^N - 1) relation subsets using bitmasks (1 << i). Memoize the optimal join tree and physical algorithm (NLJ vs HashJoin) for every subset.

[SKIP / SIMPLIFY - Low ROI Toil]:
• Skip complex hyper-graph partitioning for 50+ tables (greedy heuristics). Bitmask DP for up to 8-10 tables is optimal and standard.
• Skip complex histogram bucketing (use uniform distribution assumption: sel = 1 / V(A)).

Conceptual Takeaway: *The query optimizer is the 'brain' of a relational database. It prevents catastrophic factorial O(N!) join order blunders and random disk I/O thrashing.*

Validation Test: OptimizerDPTest: Selectivity crossover (0.1% picks IndexScan, 40% picks SeqScan), filter pushdown, and 3-way/4-way DP join order selection.
Deliverable Checkpoint: Cost-based query planner selecting optimal access paths and DP join orders based on I/O cost estimation.

SPRINT 1.5: CONCURRENCY CONTROL: MVCC SNAPSHOT ISOLATION — CORE PRIORITY

Prerequisite Lectures: CMU 15-445 Lecture #17, #18, #20 (Concurrency Control & MVCC)
Primary Learning Goal: Implement Multi-Version Concurrency Control so readers never block writers and writers never block readers.

[BUILD - High ROI Core]:
• Tuple Version Header: Augment `UserRow` with int64_t xmin (creating transaction ID) and int64_t xmax (deleting transaction ID, 0 if alive).
• Transaction Manager: Assigns monotonically increasing `txn_id`. Maintains active_txn_set.
• Snapshot Creation: When transaction T begins, capture Snapshot = {T.id, active_txns}.
• Visibility Rule: A row is visible to transaction T if:
1. row.xmin <= T.id AND row.xmin was committed when T started.
2. row.xmax == 0 OR row.xmax > T.id OR row.xmax belongs to an active uncommitted transaction.
• Non-Blocking Reads: SeqScan and IndexScan check visibility without taking shared read locks!

[SKIP / SIMPLIFY - Low ROI Toil]:
• Skip multi-granularity hierarchical lock escalation (IS/IX/S/X tables). MVCC provides superior concurrency with simpler mechanics.
• Skip out-of-line rollback segments / undo log pointer chains. Use in-place append-only version records for initial validation.

Conceptual Takeaway: *MVCC is the foundation of PostgreSQL, Oracle, and MySQL InnoDB. Understanding snapshot visibility eliminates 99% of concurrency bugs.*

Validation Test: MVCCConcurrencyTest: Spawn 1 writer thread continuously updating rows and 5 reader threads continuously calculating `SUM(score)`; assert readers see consistent snapshots with zero lock pauses.
Deliverable Checkpoint: Non-blocking MVCC query execution engine with snapshot isolation.

SPRINT 1.6: WRITE-AHEAD LOGGING (WAL) & REDO CRASH REPLAY — CORE PRIORITY

Prerequisite Lectures: CMU 15-445 Lecture #21 & #22 (Write-Ahead Logging & Crash Recovery)
Primary Learning Goal: Guarantee ACID Durability using the core WAL flush invariant without drowning in complex ARIES undo CLRs.

[BUILD - High ROI Core]:
• LogRecord: Binary struct {lsn_t lsn, txn_id_t txn_id, LogType type, RID rid, UserRow before_image, UserRow after_image}.
• LogManager: In-memory append-only ring buffer + background flusher thread flushes log buffer to disk.
• **The Core WAL Invariant**: In BufferPoolManager::FlushPage(page_id), enforce:
page->GetLSN() <= log_manager->GetFlushedLSN().
If the page has an unflushed LSN, the log buffer must be flushed to disk before writing the 4KB data frame!
• Group Commit: Batch multiple transaction commit flushes into a single synchronous disk write.
• Crash Recovery (Redo-Only): On database restart, scan the WAL file forward from start to end; for any committed transaction whose record.lsn > page->GetLSN(), re-apply the after_image to the page.

[SKIP / SIMPLIFY - Low ROI Toil]:
• Skip the full ARIES 3-pass Undo phase with Compensation Log Records (CLRs). Undo logic across arbitrary crashes is notoriously brittle.
• Skip non-quiescent fuzzy checkpointing during initial implementation (use simple clean sync checkpoints).

Conceptual Takeaway: *The WAL invariant is the single most important rule in database durability: log records must precede dirty pages to disk.*

Validation Test: CrashRecoveryTest: Insert 5,000 rows, commit, kill process with kill -9 before dirty buffer pages are flushed to disk; restart engine, invoke Redo recovery, and assert all 5,000 rows exist.
Deliverable Checkpoint: Crash-resilient database storage engine satisfying ACID Durability.

SPRINT 1.7: INTERACTIVE SQL SERVING (POSTGRES WIRE PROTOCOL OR CLI) PHASE 1 CAPSTONE

Prerequisite Lectures: CMU 15-445 Lecture #14 & CMU 15-721 Lecture #12 (Database Networking)
Primary Learning Goal: Turn your storage and execution engine into an interactive server accessible via standard terminal tools.

[BUILD - High ROI Core]:
• Upgrade threaded_server.c to parse basic PostgreSQL Wire Protocol v3.0 messages:
• • Handshake: Respond to StartupMessage with AuthenticationOk ('R') and ReadyForQuery ('Z').
• • Query: Receive 'Q' (raw query string), dispatch to parser/executor, return RowDescription ('T'), DataRow ('D'), and CommandComplete ('C').
• Alternative (Even Leaner): Build a clean interactive REPL terminal CLI (like SQLite's command prompt) that parses simple commands: INSERT, SELECT, JOIN, and EXPLAIN.

[SKIP / SIMPLIFY - Low ROI Toil]:
• Do not implement SSL/TLS encryption, complex scram-sha-256 authentication, or extended query protocol (Parse/Bind/Execute phases).

Conceptual Takeaway: *Connecting real clients (like `psql` or Python scripts) proves that your database is a real system, not just an academic library.*

Validation Test: PsqlConnectionTest: Run psql -h 127.0.0.1 -p 5432 and execute interactive queries successfully.
Deliverable Checkpoint: Interactive database server reachable by standard database client tools.

4. PHASE 2: MODERN OLAP & VECTORIZED EXECUTION (CMU 15-721)

The Analytical Revolution: SIMD, Columnar Storage & DuckDB Architecture

In modern data processing, disk I/O is no longer the sole bottleneck. With NVMe SSDs streaming at 7 GB/s, the CPU and memory bus become the primary choke points. CMU 15-721 teaches how modern engines (DuckDB, ClickHouse, Snowflake) process **billions of rows per second** by replacing tuple-at-a-time Volcano execution with **columnar vector batches**.

SPRINT 2.1: COLUMNAR STORAGE & DICTIONARY ENCODING PHASE 2 (HIGH ROI)

Prerequisite Lectures: CMU 15-721 Lecture #01, #02, #03 (Modern OLAP & Data Formats)
Primary Learning Goal: Store data column-by-column rather than row-by-row to maximize CPU cache utilization during analytical aggregations.

[BUILD - High ROI Core]:
• Columnar Chunk Representation: Store each column as a contiguous array: int64_t ids[], double scores[].
• Dictionary Encoding: Replace repetitive strings with 1-byte or 2-byte integer codes mapped to a unique string dictionary.
• Run-Length Encoding (RLE): Compress sorted or repetitive columns into (value, run_length) pairs.
• Analytical Scan Benchmark: Compute SUM(score) by scanning only the score column, skipping all other attributes entirely!

[SKIP / SIMPLIFY - Low ROI Toil]:
• Skip full Apache Parquet format decoders (Parquet involves complex Thrift metadata and nested Dremel repetition levels).
• Implement in-memory columnar chunks (Arrow RecordBatch style).

Conceptual Takeaway: *Columnar storage allows analytical queries to read only the columns they need, reducing memory bus traffic by 80–90%.*

Validation Test: ColumnarScanBenchmark: Compare calculating SUM(score) over 10 million rows in Row format vs Columnar format; measure memory bandwidth and cache misses.
Deliverable Checkpoint: High-performance columnar storage layout with dictionary compression.

SPRINT 2.2: VECTORIZED BATCH EXECUTION & SIMD PROCESSING (THE DUCKDB MODEL) PHASE 2 (CROWN JEWEL)

Prerequisite Lectures: CMU 15-721 Lecture #04, #05, #06 (Vectorized Execution Using SIMD)
Primary Learning Goal: Replace Volcano tuple-at-a-time execution with 1024-element vector batch processing to achieve 50x-100x speedup.

[BUILD - High ROI Core]:
• The Vectorized Iterator: Instead of bool Next(Tuple* out), operators implement:
int Next(VectorBatch* batch) where VectorBatch holds arrays of 1024 values per column.
• Selection Vectors: Represent filter results as an array of active row offsets (e.g. uint16_t sel_vector[]), completely eliminating data copying during filter passes.
• SIMD Vector Primitives (AVX2):
• Use AVX2 intrinsics (_mm256_cmpgt_epi32) to evaluate filter conditions (e.g. WHERE age > 30) at 8 integers per CPU cycle.
• Use horizontal SIMD additions (_mm256_add_epi32) for aggregations (`SUM`, `AVG`).

[SKIP / SIMPLIFY - Low ROI Toil]:
• Skip LLVM IR machine code generation (JIT compilation). Vectorized execution delivers ~95% of compiled performance with 1/10th the complexity.
• Skip dynamic JIT template engines (DuckDB proved vectorized interpretation beats JIT for interactive queries).

Conceptual Takeaway: *This is the core design pattern that made DuckDB and ClickHouse the fastest embedded analytical engines in the world.*

Validation Test: VectorizedVsVolcanoBenchmark: Run WHERE score > 50.0 over 20,000,000 records; observe Volcano takes ~600ms while Vectorized execution takes ~8ms (a 75x speedup!).
Deliverable Checkpoint: Vectorized execution engine processing hundreds of millions of values per second per core.

SPRINT 2.3: MORSEL-DRIVEN WORK-STEALING PARALLELISM PHASE 2 (HIGH ROI)

Prerequisite Lectures: CMU 15-721 Lecture #08 (Query Scheduling & Coordination)
Primary Learning Goal: Scale query execution linearly across all available CPU cores using dynamic morsel-driven scheduling.

[BUILD - High ROI Core]:
• Morsel Dispatcher: Partition input table scans into dynamic, fixed-size chunks ('morsels' of 10,000 to 50,000 rows).
• Lock-Free Work-Stealing Thread Pool: Worker threads dynamically pull morsels from queues, stealing from neighbor queues when idle to eliminate straggler cores.
• Thread-Local Aggregations: Each thread accumulates partial aggregate hash tables, followed by a fast parallel reduction merge.
• Multicore Scaling Test: Measure speedup scaling from 1 core to 2, 4, 8, and 16 CPU cores.

[SKIP / SIMPLIFY - Low ROI Toil]:
• Skip complex hardware NUMA node binding configurations (focus on dynamic work-stealing thread pools first).
• Skip distributed shuffle networks (keep execution single-node multicore).

Conceptual Takeaway: *Static query partitioning causes severe imbalances when data is skewed. Morsel-driven execution guarantees 100% CPU core utilization.*

Validation Test: ParallelScalingBenchmark: Run analytical query on 50 million rows across 1, 2, 4, and 8 threads; verify near-linear scaling curve.
Deliverable Checkpoint: Elastic multicore parallel query scheduler with work-stealing load balancing.

5. STREAMLINED LECTURE SYNCHRONIZATION MATRIX

CMU 15-445 & 15-721 Action Guide: What to Watch, What to Build, What to Skip

Use this matrix as your tactical study roadmap. Notice the clear distinction between lectures that require code versus lectures that are purely conceptual or can be skimmed:

Course & Lecture	Lecture Title	Key Architectural Concept	Action: Build, Simplify, or Skip
15-445 #01-02	Relational Model & Modern SQL	Relational algebra, set operations.	SKIM (Theory only)
15-445 #03	Database Storage: Files, Pages, Tuples	Slotted page layout, Record IDs (RID).	**BUILD (Sprint 1.2: Fixed TableHeap)**
15-445 #04	Memory Management & Buffer Pools	Frame cache, LRU-K, pin/unpin.	**COMPLETED (Your BPM is done!)**
15-445 #05	Log-Structured Storage (SingleStore)	LSM-Trees, MemTable, SSTables.	WATCH (Skip coding for now)
15-445 #06	Column-Store Databases	PAX format, analytical decomposition.	**WATCH (Prepares for Phase 2)**
15-445 #07	Database Hash Tables (YugabyteDB)	Hash tables for joins and aggs.	**BUILD (Sprint 1.3: Hash Join)**
15-445 #08	B+Trees: The Best Data Structure	Node splits, merges, tree depth.	**COMPLETED (Your B+ Tree is done!)**
15-445 #09	Vector, Inverted Indexes & Filters	Bloom filters, HNSW vector search.	SKIM (Optional for now)
15-445 #10	Latching in Data Structures	Read/Write latch crabbing, coupling.	**COMPLETED (Milestone 7-8 done!)**
15-445 #11	Sorting & Aggregation Algorithms	External merge sort, hashing aggs.	**SIMPLIFY (In-memory sort & aggs)**
15-445 #12	Joins: Hash, Sort-Merge, Nested Loop	Hash join build/probe algorithms.	**BUILD (Sprint 1.3: Hash Join)**
15-445 #13	Query Execution Part 1	Volcano iterator model (Init/Next).	**BUILD (Sprint 1.3: Volcano Engine)**
15-445 #14	Parallel Query Engine Architectures	Process models, thread pools.	**WATCH (Prepares for Server / Morsels)**
15-445 #15-16	Query Optimization Part 1 & 2	Cost models, Access path selection.	**BUILD (Sprint 1.4: Access Path Cost)**
15-445 #17-19	Concurrency Theory & 2-Phase Locking	Serializability, 2PL, deadlocks.	**WATCH (Study theory, jump to MVCC)**
15-445 #20	MVCC: Multi-Version Concurrency	Snapshot isolation, xmin/xmax headers.	**BUILD (Sprint 1.5: Snapshot Reads)**
15-445 #21-22	WAL Logging & Crash Recovery (ARIES)	WAL invariant, Analysis/Redo passes.	**BUILD (Sprint 1.6: Redo-Only WAL)**
15-445 #23-25	Distributed Databases & Cloud Warehouses	2PC, consensus, distributed queries.	WATCH (High-level systems review)
15-721 #01	Modern OLAP Database Systems	Architecture of analytical engines.	**WATCH (High priority for Phase 2)**
15-721 #02-03	Data Formats & Encoding Part 1 & 2	Dictionary, Bitpacking, FSST.	**BUILD (Sprint 2.1: Columnar Chunks)**
15-721 #04-05	Query Execution & Processing	Vectorized vs Push compilation.	**BUILD (Sprint 2.2: Vector Batch)**
15-721 #06	Vectorized Execution Using SIMD	AVX2/AVX-512 filter bitmasks.	**BUILD (Sprint 2.2: SIMD Filters)**
15-721 #07	JIT Compilation & Code Generation	HyPer code gen vs DuckDB vectorization.	**WATCH ONLY (Skip raw LLVM IR coding)**
15-721 #08	Query Scheduling & Coordination	Morsel-driven work-stealing parallelism.	**BUILD (Sprint 2.3: Morsel Scheduler)**
15-721 #09-10	Parallel Hash Joins & WCOJ	Radix hash join, Leapfrog Triejoin.	SKIM (Niche graph theory; skip WCOJ)
15-721 #12-15	Networking & Cascades Optimizer	Arrow Flight, Memo optimizer search.	**WATCH (Concept study)**
15-721 #17-22	DuckDB, Photon, Snowflake, BigQuery	Case studies of commercial cloud engines.	WATCH (Excellent case studies)

6. TOOLCHAIN, BENCHMARKING & IMMEDIATE ACTION PLAN

Verification Standards and the Next 7 Days

To ensure that your implementation is industrial-grade and crash-resilient, apply these exact compiler sanitizers and benchmarking tools throughout your development:

- **AddressSanitizer (ASan) & UndefinedBehaviorSanitizer (UBSan):** Always compile with -fsanitize=address,undefined -g -O1. Catches buffer overflows, out-of-bounds reads in table pages, and dangling pointer dereferences.
- **ThreadSanitizer (TSan):** Compile multi-threaded MVCC reader/writer and morsel scheduler tests with -fsanitize=thread -g -O1. Guarantees zero data races on shared page memory.
- **Microbenchmarking with Perf Counters:** Use perf stat -e instructions,cycles,L1-dcache-load-misses,branch-misses. When you compare Volcano vs Vectorized execution in Sprint 2.2, you will directly see IPC jump from ~0.8 to >2.5!

Target Benchmarks for Your Engine:

Benchmark	Query Shape	Primary Metric	What It Proves
Point Index Scan	WHERE id = 42	Latency (µs)	Validates B+ Tree O(log N) traversal fetching matching RID from TableHeap in <5 microseconds.
Hash Join	Users JOIN Orders ON u.id = o.user_id	Throughput (rows/sec)	Validates that in-memory hash table build/probe outperforms nested loops by 100x+.
MVCC Snapshot Read	Concurrent table scan while 10 threads update rows.	Zero lock stalls	Proves that readers never block writers and writers never block readers.
Vectorized Scan (OLAP)	SELECT SUM(score) WHERE score > 50	GB/s scan bandwidth	Demonstrates the 50x speedup of 1024-batch SIMD execution over Volcano.

Immediate Action Plan: Sprint 1.2

Your Immediate Next Step (Next 48 Hours):
1. Open CMU 15-445 **Lecture #03 (Database Storage: Files, Pages, Tuples)**.
2. Create src/storage/table/table_page.h and src/storage/table/table_heap.h.
3. Define a fixed struct: struct UserRow { int64_t id; char name[32]; double score; int64_t xmin; int64_t xmax; };.
4. Implement TableHeap::Insert(row, &rid;) and TableHeap::Get(rid, &row;) over your existing BufferPoolManager.
5. Connect your B+ Tree: insert (row.id, rid) into the tree, search for row.id, fetch the row from TableHeap, and verify it matches!

You will have a working indexed table storage engine in a single coding session.